

# Cleaning Schedule Add-on — System Retrospective

Generated 2026-05-06 · Version 1.17.x · Working dir: `cleaning-schedule-addon/`

## 1. Purpose & Philosophy

A **Home Assistant add-on** that tracks Airbnb property cleaning schedules. It syncs bookings from an Airbnb iCal feed, assigns cleaners to checkout dates, and uses **Claude Haiku** to interpret WhatsApp conversations with cleaners — detecting confirmations, declines, and time proposals automatically.

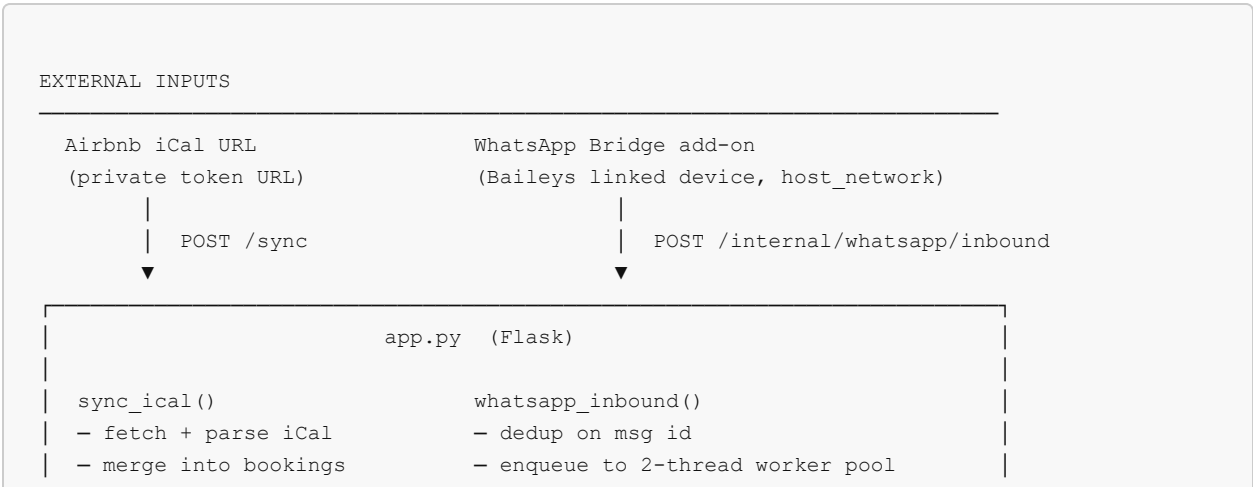
**Design principle:** the add-on is the *brain*; Google Calendar is the *shared view*. `data.json` is the single source of truth. GCal is a one-way projection for Michelle and the cleaners to see. The UI handles everything GCal can't: per-cleaner notify queues, WhatsApp review, and conflict detection.

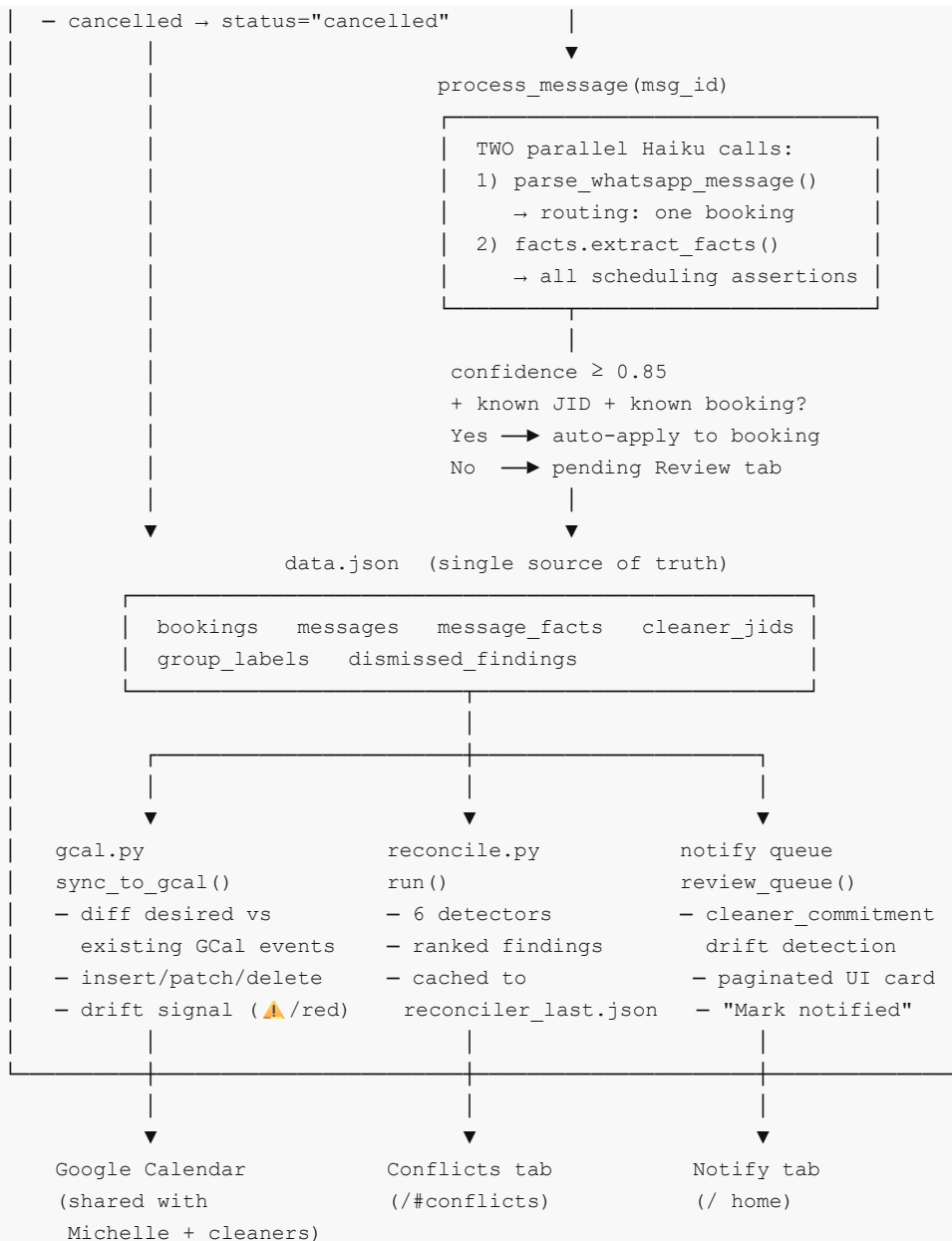
### The four data sources the reconciler joins

- **Airbnb iCal** — the booking calendar (fetched live).
- **data.json bookings** — local state: assignments, cleaner commitments, custom entries.
- **Google Calendar** — the projected shared view (compared for drift).
- **WhatsApp archive** — message facts extracted by Claude Haiku.

NOTES

## 2. System Flow Diagram





## ALTERNATE INGEST PATHS

|                                                        |                                     |
|--------------------------------------------------------|-------------------------------------|
| /backfill (GET/POST)                                   | /admin/ingest-transcript (POST)     |
| - paste WhatsApp export                                | - paste raw transcript text         |
| - Haiku returns per-booking proposals (cleaner + time) | - parse into messages log           |
| - review page → apply                                  | - run facts extraction              |
| - ack_notified(via="backfill")                         | - idempotent (stable ids)           |
|                                                        | - progress via /admin/ingest-status |

## ADMIN / MAINTENANCE

|                         |                                          |
|-------------------------|------------------------------------------|
| /admin/reprocess-facts  | re-extract stale-prompt messages (batch) |
| /admin/fix-parse-errors | reset 429-errored messages → pending     |
| /admin/remap-group      | bulk rewrite group JIDs + labels         |
| /reconcile/dismiss      | human marks finding resolved out-of-band |
| /digest/run             | trigger daily digest → HA notification   |

NOTES

### 3. File Map

---

#### ***cleaning-tracker/app.py (~3,076 lines) — the whole application***

Single-file Flask app. Every route, template (inline Jinja2 strings), business logic, worker threads, and helper functions live here. Key areas:

- **Bootstrap** (lines 1–200): `load_options()`, `load_data()`, `save_data()`, `ingress_prefix()`
- **iCal sync**: `sync_ical()`
- **Haiku parse**: `parse_whatsapp_message()`
- **Worker pool**: `enqueue_message()`, `_message_worker()`, `process_message()`
- **Notify queue**: `review_item()`, `needs_notify()`, `review_queue()`, `ack_notified()`, `build_focus_context()`
- **Reconciler glue**: `_fetch_ical_events()`, `_run_full_reconcile()`, `_rerun_reconcile_cached()`
- **Digest**: `_digest_compute_and_notify()`, `_digest_scheduler()`
- **Transcript ingest**: `_parse_whatsapp_transcript()`, `_ingest_worker()`
- **Backfill**: `_backfill_haiku()`, routes `/backfill` + `/backfill/apply`
- **Print view**: `build_print_data()`, `/print`

#### ***cleaning-tracker/facts.py (207 lines) — structured fact extraction***

Separate Haiku call that extracts *every* scheduling assertion in a message (confirm, decline, time\_proposal, date\_proposal, schedule\_assertion, unclear). Results stored in `data.message_facts`, versioned by `FACTS_PROMPT_VERSION` (currently `facts-v2`).

- `extract_facts()` — calls Haiku with role-tagged history (host/cleaner), retries 429/5xx with exponential backoff.
- `_facts_history()` — caps at last 30 messages from the same group to avoid TPM budget exhaustion.
- `build_record()` — shapes raw Haiku output into the stored record format.

#### ***cleaning-tracker/gcal.py (361 lines) — Google Calendar projection***

One-way sync: `data.json` → `GCal`. Never reads `GCal` to make decisions — only to diff for sync.

- `sync_to_gcal()` — diffs desired events against tagged existing events; insert/patch/delete. Module-level lock prevents race-condition duplicates.
- `_desired_events()` — builds the target event list. Drift bookings get `colorId=11` (red) + ⚠️ prefix.
- `fetch_tagged_events()` — used by reconciler to snapshot `GCal` state.
- All events tagged with `extendedProperties.private.source="cleaning-tracker"`.
- Auth: service account JSON (no OAuth, no refresh tokens).

***cleaning-tracker/reconcile.py (494 lines) — pure-function detectors***

Takes `data.json` + iCal events + GCal events → returns ranked findings list. No side effects; all writes happen in `app.py`.

- `run()` — calls all 6 detectors, deduplicates by stable id, sorts by severity.
- `filter_and_sort()` — strips dismissed findings (used after dismiss/undismiss without re-fetching).

NOTES

---

---

---

---

## 4. Data Model ( /data/data.json )

No database. Everything lives in one JSON file. Lazily backfilled on read — all fields are additive.

### *bookings (keyed by UID)*

- `type`: "airbnb" | "custom\_stay" | "manual\_cleaning"
- `start`, `end`: ISO dates. `end` is exclusive (Airbnb convention).
- `status`: "active" | "cancelled" | "complete"
- `cleaner`: assigned cleaner name or null.
- `clean_time`: "HH:MM:SS" or null. Shown in calendar titles.
- `cleaner_commitment`: snapshot of what the cleaner was told: {`cleaner`, `date`, `clean_time`, `communicated_at`, `communicated_via`}. Absent on legacy/freshly-assigned bookings → appears in notify queue as "new". Drift between this and current truth drives the notify queue and GCal ⚠️ signal.
- `notes`: free-text (legacy "Time: HH:MM AM" prefix auto-backfilled into `clean_time`).

### *messages (inbound WhatsApp log)*

Each entry: {`id`, `timestamp`, `sender`, `group`, `text`, `parsed`, `applied_uid`, `review_state`, `haiku_result?`, `source?`, `sender_name_raw?`}

- `review_state`: "auto" (applied automatically) | "pending" (needs human review) | "ignored"
- `source`: "backfill" + `haiku_result.backfill_ingest=true` = facts-only ingest, never shown in Review tab.

### *message\_facts (keyed by message id)*

{`facts`: [...], `reported_by_jid`, `model_version`, `prompt_version`, `extracted_at`}

Each fact: {`kind`, `target_date`, `target_time`, `cleaner`, `confidence`, `tentative`, `evidence`}

Kinds: `confirm` `decline` `time_proposal` `date_proposal` `schedule_assertion` `unclear`

Reconciler reads only records matching `FACTS_PROMPT_VERSION` — half-reprocessed state is safe.

### *Other top-level keys*

- `cleaner_jids`: {`jid`: `cleaner_name`} — built from Review-tab mappings.
- `group_labels`: {`group_jid`: `label`} — human-readable group names for the UI.
- `dismissed_findings`: {`finding_id`: {`dismissed_at`, `reason`}} — reconciler findings a human marked resolved.

**Separate file:** /data/reconciler\_last.json — cached reconciler output (not in `data.json`).

Stores `findings_raw` (pre-dismiss) and `findings` (post-filter).

NOTES

## 5. UI Routes & Pages

---

### / (home — three tabs)

- **Notify tab** (default): One cleaner at a time. Cards show bookings whose `cleaner_commitment` diverges from truth. Kinds: *new* (no commitment), *changed* (drifted), *cancelled* (post-commitment cancellation). Unassigned bucket shown separately. `?i=N` paginates cleaner buckets. "Mark notified" writes commitment for the whole cleaner-batch.
- **WhatsApp Review tab** (`#review`): Pending messages. Accept / override / ignore. Map unknown senders to cleaners. Label groups.
- **Conflicts tab** (`#conflicts`): Cached reconciler findings grouped by severity. One-click: Assign, Edit booking, Dismiss. Badge = needs-attention count.

### Booking management

- `/add` — radio: Cleaning vs Stay. Stays need start+end; cleanings take one date + optional cleaner + time.
- `/edit/<uid>` — edit cleaner, notes, `clean_time`. Cancelled → Dismiss button. Custom/manual → Delete.
- `/assign/<uid>` — write cleaner + `clean_time` (clears if blank).
- `/delete/<uid>` — soft delete (marks cancelled or removes from data).
- `/sync` (POST) — trigger iCal sync manually.
- `/gcal/sync` (POST) — trigger GCal sync manually.

### Backfill & ingest

- `/backfill` — paste a WhatsApp chat export. Haiku returns per-booking proposals. Review + apply page.
- `/admin/ingest` — paste raw transcript. Parses into messages log, runs facts extraction. Progress via `/admin/ingest-status`.

### Print & utility

- `/print?month=YYYY-MM` — hand-rolled HTML table, print-optimized CSS. Shows stays (colour bars) + cleaner+time on checkout cells.
- `/internal/snapshot` — returns full `data.json` + reconciler cache (used by off-host reconcile script).
- `/digest/run` — trigger daily digest manually. Posts HA notification.

### Reconcile routes

- POST `/reconcile/run` — recompute + persist. Accepts form post (redirects) or JSON.
- GET `/reconcile/last` — serve cached JSON.
- POST `/reconcile/dismiss` + `/reconcile/undismiss` — toggle dismissed state, re-filter cache.



NOTES

## 6. WhatsApp Pipeline

### 6a. WhatsApp Bridge add-on ( *whatsapp-bridge/* )

A separate HA add-on running **Baileys** (Node.js). Pairs as a WhatsApp linked device via QR scan.

- Uses `host_network: true` → reaches cleaning tracker at `http://127.0.0.1:5000` (loopback, no shared secret needed).
- Forwards **all group messages** including host's own outbound ( `fromMe: true` is NOT filtered — both sides reach the facts layer).
- Read-only: never calls `sendMessage`.
- Filters: non-group dropped, group not in `group_allowlist` dropped, empty text dropped.
- Auth state persists in `/data/auth/` across restarts. Subsequent starts reconnect automatically.
- On first connect: logs all visible groups with JIDs → use Log tab to populate `group_allowlist`.
- Startup backfill option ( `backfill_per_group` ): returns zero in practice on reconnects — WhatsApp doesn't replay history for caught-up devices. Use transcript ingest for deep history.

### 6b. Live inbound pipeline

```
POST /internal/whatsapp/inbound
├ dedup: id already in messages? → 200 early-return
├ auth: loopback? ✓  non-loopback needs X-Shared-Secret header
├
└ enqueue_message(msg_id) → 2-thread worker pool
    └ process_message(msg_id)
        ├── _parse_history() → last N messages (full history, same group)
        ├── _facts_history() → last 30 messages (same group)
        ├──
        ├── [Thread A] parse_whatsapp_message() ← routing Haiku call
        │   Input: message + history + bookings + cleaners + labels
        │   Output: {uid, cleaner, action, confidence, evidence}
        ├──
        └ [Thread B] facts.extract_facts() ← facts Haiku call
            Input: message + 30-msg history + cleaners + labels
            Output: [{kind, target_date, cleaner, confidence, ...}]
            Stored → data.message_facts[msg_id]
```

After both calls:

```
confidence ≥ 0.85 AND known cleaner JID AND known booking?
YES → _apply_booking_change() → auto review_state="auto"
NO  → review_state="pending" → appears in Review tab
```

**6c. Backfill flow ( `/backfill` )**

- User pastes WhatsApp export. Haiku receives transcript + all active unassigned bookings + known cleaners + optional group hint.
- Returns per-booking proposals: `{uid, cleaner, clean_time, confidence, evidence}`.
- Review page: confidence-coloured borders (green  $\geq 0.85$  / yellow  $0.6-0.85$  / red  $< 0.6$ ), editable fields, checkbox pre-ticked at  $\geq 0.85$ .
- On apply: writes assignments, calls `ack_notified(via="backfill")` → skips notify queue.

**6d. Transcript ingest ( `/admin/ingest-transcript` )**

- Parses three timestamp formats: `[YYYY-MM-DD, HH:MM:SS AM/PM]`, `[H:MM AM/PM, M/D/YYYY]`, Android `YYYY-MM-DD, H:MM a.m./p.m. - Sender: text`.
- Stable ids ( `backfill-sha1(ts|sender|text)[:16]` ) — re-runs are idempotent and dedup against live messages.
- Runs facts extraction (or full `process_message` if `apply=true`) in a background thread.
- Progress polled via `/admin/ingest-status`.

NOTES

---

---

---

---

## 7. Facts Layer ( `facts.py` )

The facts layer answers a different question than the parse layer. **Parse** asks: "route this single message to one booking?" **Facts** asks: "list every date/cleaner/time assertion in this message."

### *Fact kinds*

- `confirm` — cleaner confirms a date (cleaner-only)
- `decline` — cleaner declines a date (cleaner-only)
- `time_proposal` — cleaner proposes a time (cleaner-only)
- `date_proposal` — cleaner proposes an alternative date (cleaner-only)
- `schedule_assertion` — host states who cleans what date (host-only)
- `unclear` — ambiguous

### *Key design decisions*

- **Role-tagged history:** each history line labelled `<host>` or `<cleaner:Name>`. Haiku uses role to determine allowed fact kinds (e.g. host can't produce a `confirm`).
- **Re-posted-list recognition:** when a cleaner re-posts a full schedule list, it's treated as re-confirmation of all listed dates. This is the dominant real-chat pattern.
- **History window (30 messages):** only last 30 messages from the same group are sent. Prevents TPM budget exhaustion on bulk reprocess (a previous stalled ingest hit 284/952 over 5 hours before the cap was added).
- **Versioning:** `FACTS_PROMPT_VERSION = "facts-v2"` stamps every record. Reconciler reads only current-version records — safe to bump and reprocess incrementally.
- **Rate-limit handling:** exponential backoff honouring `retry-after`. Bulk ingest paced at 0.8s/call.

NOTES

---



---



---



---

## 8. Reconciler ( `reconcile.py` )

Pure-function. Takes data as arguments, returns findings. No side effects. All 6 detectors are shipped.

### *Severity tiers*

- **needs-attention** — drift, decline-still-assigned, contested cleaner, schedule mismatch
- suggest — unrecorded confirmation, schedule vs unassigned booking
- informational — confirm with no booking, changed-mind timeline

## Detectors

- **\_ical\_vs\_bookings:** Airbnb iCal ↔ bookings. Emits: `ical_missing_booking`, `booking_not_in_ical`, `ical_date_mismatch`, `ical_resurrected`.
- **\_bookings\_vs\_gcal:** bookings ↔ GCal. Emits: `gcal_missing_event`, `gcal_stale_event`, `gcal_orphan`. Only runs when `gcal_enabled`.
- **\_drift:** reshapes notify queue into findings (new / changed / cancelled / unassigned).
- **\_facts\_vs\_bookings:** confirm/decline facts ↔ booking state. Emits: `unrecorded_confirmation`, `contested_cleaner`, `decline_still_assigned`, `confirm_no_booking`.
- **\_fact\_timeline:** `changed_mind` when a cleaner said both confirm and decline on the same date (latest wins).
- **\_schedule\_vs\_bookings:** host `schedule_assertion` ↔ booking cleaner. Emits: `schedule_mismatch`, `schedule_unassigned`.

## Known issues / noise

- Detector 6 ( `_schedule_vs_bookings` ) doesn't collapse stale host assertions. Old host messages generate `schedule_mismatch` noise. A latest-wins pass over `(cleaner, target_date)` by message timestamp would fix it.
- Facts dedup: nothing collapses duplicate assertions across messages. Reconciler groups by `(cleaner, target_date)` in `_fact_timeline` + `_schedule_vs_bookings` but not across detectors.

NOTES

---

---

---

---

## 9. Google Calendar Projection (`gcal.py`)

---

### *Philosophy*

GCal is a **read-only view for humans**. The add-on never reads GCal to make decisions — only to diff for sync. Cleaners confirm via WhatsApp, not by editing the calendar.

### *How sync works*

- `sync_to_gcal()` builds the desired event list, lists existing events tagged with `source="cleaning-tracker"`, then insert/patch/deletes to converge.
- **Serialized** with a module-level lock — concurrent calls skip and return `{"skipped": 1}`. Prevents race-condition duplicates when `save_data()` fires many times in a row during iCal sync.
- **Dedupes on the fly:** `_list_existing` deletes any duplicate events (same UID tag, multiple events) at the start of each sync.
- Triggered via `save_data()` in a daemon thread (fire-and-forget; GCal outages don't block local writes).

### *Event rules*

- **Cleaning events always timed** (never all-day). Default 11:00 AM–1:00 PM when `clean_time` unset. Otherwise from `clean_time` for 2 hours.
- **Timezone:** `America/Vancouver` (constant `LOCAL_TZ` in `gcal.py`).
- **Drift signal:** cleaning events with unresolved drift get `colorId=11` (red) + 🚩 prefix. Resolves on next sync after "Mark notified".
- **Cleaner colour:** md5-hashed onto 9 palette slots (slot 8 reserved for cancelled, 11 for drift/unassigned).
- **Cancelled bookings:** omitted from desired events (deleted from GCal) *unless* the cleaner has an unacknowledged commitment — then a red ✖ Cancelled event is kept until commitment cleared.
- **Auth:** service account JSON. No OAuth, no refresh-token expiry. Share calendar with service account email at "Make changes to events".

NOTES

---



---



---



---



---

## 10. Notify Queue Logic

---

### *What drives the queue*

A booking enters the queue when its `cleaner_commitment` diverges from current truth. The "truth tuple" is `(cleaner, checkout_date, clean_time)`. The "commit tuple" is the same fields from `cleaner_commitment`.

- **new**: cleaner assigned, no commitment exists.
- **changed**: commitment exists but one of the three fields differs.
- **cancelled**: booking cancelled after commitment was written.
- **unassigned**: active Airbnb booking with no cleaner at all → separate bucket at top of page.

### *Resolution*

"Mark notified" (`POST /review/notify/<slug>`) calls `ack_notified(booking, via="manual")` for every booking in the cleaner's bucket. This writes a fresh `cleaner_commitment` matching current truth, removing the drift. Unit of work is **one cleaner = one WhatsApp message**. No per-line ticking.

WhatsApp auto-apply writes `communicated_via="whatsapp"`; backfill apply writes `communicated_via="backfill"`. Both have the same effect as "Mark notified".

NOTES

---

---

---

---

## 11. Auth & Deployment

---

### *Auth model ( `_require_local_or_secret()` )*

Used for all `/admin/*`, `/internal/*`, and `/reconcile/*` routes. Accepts any of:

- **Loopback** — `127.0.0.1` (WhatsApp Bridge calls, local scripts).
- **HA ingress** — presence of `X-Ingress-Path` header (Supervisor proxy stamps it; the browser uses this path).
- **Shared secret** — `X-Shared-Secret` matching `whatsapp_shared_secret` option (off-host callers).

### *Deployment*

- Installed via HA custom repository URL. Add-on Store → Repositories → paste URL → Install.
- Updates: bump `version` in `config.yaml`, push to GitHub, refresh in HA. **Always bump on every push** — Supervisor caches configs.
- `init: false` in `config.yaml` required (s6-overlay conflict with bare `CMD`).
- Port 5000 exposed on LAN for off-host tooling ( `reconcile_pull.py`, etc.).
- Dockerfile: `python:3.12-slim` hardcoded (HA's `BUILD_FROM` arg pattern didn't work — Supervisor wasn't passing it through).
- Do NOT use Samba for development on HAOS from Windows — SMB write caching makes files stale.

### *Add-on options*

- `ical_url` — Airbnb iCal URL (private token — never commit)
- `anthropic_api_key` — Claude Haiku key (password type)
- `cleaners` — list of cleaner names (used for dropdowns + JID mapping)
- `gcal_enabled`, `gcal_calendar_id`, `gcal_service_account_json`
- `whatsapp_shared_secret` — only needed for non-loopback callers

NOTES

---

---

---

---



12. Open Questions & Deferred Work

| Item                                              | Status         | Blocker                                     |
|---------------------------------------------------|----------------|---------------------------------------------|
| Reconciler daily digest (Step 3)                  | Deferred       | Detector 6 noise (stale host assertions)    |
| Detector 6 latest-wins collapse                   | Not started    | —                                           |
| Facts dedup across messages                       | Deferred       | Not painful enough yet                      |
| Josh-as-host signal (jokey messages)              | Deferred       | Allowlist host in config                    |
| First-run notify-queue noise (legacy data)        | Deferred       | One-shot "trust current state" admin action |
| Playwright coverage (notify queue, Conflicts tab) | Not started    | —                                           |
| Bot account swap to SpeakOut number               | Pending        | \$125/yr registration + re-pair procedure   |
| Resolved-notify audit log                         | Rejected (MVP) | —                                           |
| Retire /print route                               | Rejected       | Michelle still uses it                      |

NOTES

### 13. Scripts (not shipped in add-on image)

- `scripts/reconcile_pull.py` — off-host puller. Hits `/internal/snapshot` on port 5000, downloads full data, runs reconciler locally. Used by the `reconcile-cleaning-schedule` Claude skill.
- `scripts/gcal_auth.py` — validates a GCal service-account key, prints the email to share with, and runs a test event insert/delete.
- `scripts/fix_parse_errors.py` — resets messages stuck in a 429-error parse state back to pending. Superseded by `/admin/fix-parse-errors` route but kept as a standalone fallback.

NOTES

## 14. General / Catch-all Notes

## NOTES